CHUNK FOR NEXT PUSH: AN ANALYSIS OF ACTION CHUNKING AND HUMAN INTERVENTION IN PUSH-T

A PREPRINT

Anthony Xu
Computer Science Department
Carnegie Mellon University
Pittsburgh, PA 15213
anthonyx@andrew.cmu.edu

Jojo Li
Computer Science Department
Carnegie Mellon University
Pittsburgh, PA 15213
yizhoul2@andrew.cmu.edu

Pranav Ram
Computer Science Department
Carnegie Mellon University
Pittsburgh, PA 15213
pram@andrew.cmu.edu

April 24, 2026

1 Introduction

1.1 Background and Task

Generalist imitation learning methods have shown strong performance in robotic manipulation from human demonstrations. A recent direction in this area is *action chunking*, where a policy predicts a sequence of k future actions instead of a single step, enabling improved planning over long horizons. In this work, we adopt the Action Chunking with Transformers (ACT) architecture [7].

We also consider the role of human intervention methods such as DAgger [5], which mitigate compounding errors in behavioral cloning by aggregating expert corrections during policy rollouts. Complementarily, mid-training human intervention allows a supervisor to correct the agent online, improving robustness and safety in regions where the policy is uncertain.

This project studies the long-horizon Push-T task (Figure 1), where an agent (shown in blue) must push a T (shown in gray) into a goal zone (shown in green). While diffusion [1] and flow matching [2] methods achieve high success rates, the effectiveness of action chunking and its interaction with DAgger-style human intervention remain underexplored; we evaluate how ACT performs in this setting and its impact on success rate.

Finally, since the creators of the ACT architecture claimed the technique could compensate for messy state data to reach high success rate when performing complex tasks, this project attempts to verify this on Push-T by comparing the results from two types of Push-T data: one containing agent coordinates and images for the T block; another containing agent coordinates and 16 corners of the T block.

1.2 Datasets

The first dataset we used was `lerobot/pusht` [3], which consisted of agent coordinates and 96×96 image observations, shown in figure 2).
The second dataset we used was `lerobot/pusht_keypoints` [4], which consisted of the agent's x and y coordinates and 16 keypoints of the T block. This gave a richer and more precise position of the T block.
(For future reference, the first dataset will be referred to as the "image-only dataset," and the second one will be referred to as the "state-only dataset.")

As we cover later in more detail, the first dataset suffers from the limitation of low quality images. Both datasets suffer from lack of data for the block in dead-zones that are either extremely far or extremely close to the goal.

We also collected our own data for DAgger using customized data collection scripts that runs on the `gym-pusht` environment built for this task (figure 1).



Figure 1: PushT Environment



Figure 2: LeRobot Image Data

2 Related Works

2.1 Data Aggregation

A foundational improvement made to imitation learning was data aggregation through human expert correction trajectories collected during policy rollouts, with the DAgger technique [5] being the most influential framework.

Classic sequence prediction problems rely on statistical assumptions that are often untrue when actions are codependent across timesteps. For example, if a policy makes a tiny mistake, it drifts into a state it has never seen in the training data and makes more mistakes, spiraling out of control. Motivated by this flaw, DAgger was invented to mitigate the compounding errors caused by these incorrect statistical assumptions in Standard Behavioral Cloning (BC).

The algorithm teaches the model how to recover from its own mistakes using consistent human intervention. Specifically, the human expert provides actions for the states that the current policy $\hat{\pi}_i$ run through, the dataset is aggregated with these new state-action pairs, and the next policy $\hat{\pi}_{i+1}$ is trained on the aggregated dataset [5], see figure 3. In this project, we used a variant of DAgger to aggregate our dataset for our policies.

Initialize $\mathcal{D} \leftarrow \emptyset$.
Initialize $\hat{\pi}_1$ to any policy in Π.
for $i = 1$ **to** N **do**
  Let $\pi_i = \beta_i \pi^* + (1 - \beta_i)\hat{\pi}_i$.
  Sample T-step trajectories using π_i.
  Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i and actions given by expert.
  Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \bigcup \mathcal{D}_i$.
  Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$.
end for
Return best $\hat{\pi}_i$ on validation.

Figure 3: DAgger Algorithm [5]

2.2 Compliant Residual DAgger

DAgger is the traditional framework used to augment training data with human intervention trajectories, but it has some drawbacks. For example, DAgger usually requires manually labeling the expert actions corresponding to each of the states that the policy met, which is very inefficient, and sometimes the human action completely overwrites the previous policy. Due to policy interference and human response latency, the resulting human trajectory can be imperfect, causing the robot to learn jerky, discontinuous, and even contradictory movements.

The **Compliant Residual DAgger** (CR-DAgger) approach [6] addresses these issues by learning a *delta correction* instead of take-over correction, allowing the robot policy to execute continuously while a human applies corrective forces through a compliant interface. Rather than retraining or fine-tuning the original policy π_{base}, CR-DAgger introduces a separate *Compliant Residual Policy* π_{res}, trained solely on correction data to predict delta poses and target forces, which are then combined with π_{base}'s outputs to produce the final trajectory.

We adopted the "delta corrections" idea from CR-DAgger to fine-tune ACT on the Push-T task, aiming to compare this method's effect with that of traditional DAgger on ACT and Push-T specifically.

2.3 Action Chunking

A more recent state-of-the-art algorithm for imitation learning, and more broadly robot learning, is action-chunking, where a policy is trained to predict from a single state a series of actions rather than just one.

This improves the robot's planning capability, which is especially useful for long-horizon manipulation tasks. The **Action Chunking with Transformers (ACT)** architecture [7], proposed in the year 2023, is a prominent framework for action chunking. As the name suggests, the core of the algorithm is the usage of a transformer as a generative model to predict a sequence of actions (see figure 4).

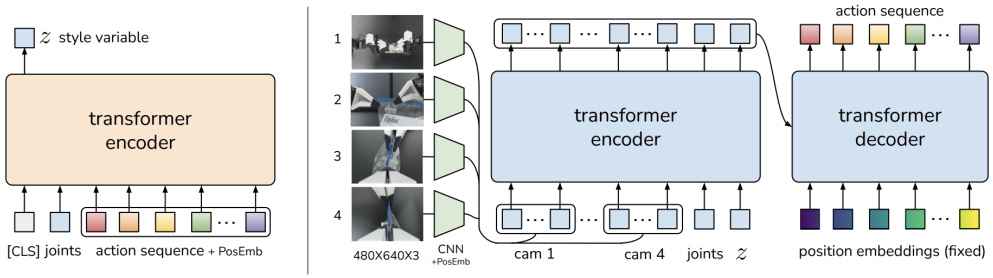


Figure 4: ACT Architecture [7]

ACT mitigates difficulty in modeling human stochasticity by introducing a latent "style variable" z via a conditional VAE. The encoder learns a distribution over z, which is sampled during training and used by the decoder to generate actions. KL divergence regularizes this latent space, with higher β enforcing stronger regularization. The transformer-based ACT architecture achieves high accuracy on benchmark tasks with limited training data and learns effectively from standard image and state observations. A major part of our project was attempting to verify the applicability of this claim to the PushT task by training on two types of dataset, one of them corresponding to image and agent states data.

2.4 Diffusion Policy

A more recent approach in robot learning is *Diffusion Policy*, which models a policy as a conditional denoising diffusion process over actions [1]. This allows the model to complete trajectories / action sequences by iteratively refining initially noisy samples conditioned on observations. At each diffusion step, the model predicts a gradient direction in which to denoise, allowing an action chunk to slowly emerge from the noise.

This enables the policy to represent action distributions without collapsing to average behaviors, similar to how the latent variable z in ACT's CVAE captures variability in example demonstrations. Different initial noises can be refined to different output action sequences, all of which are plausible.

Additionally, diffusion policies predict multi-step action sequences, similar to action chunking in ACT, which improves temporal consistency and supports long-horizon planning. Diffusion policy is proved to reach more than 90% success rate on the Push-T task, and due to a number of similarities between diffusion policy and ACT, this motivated us to explore ACT's interaction with Push-T.

3 Methods

We trained three **baseline methods** on both datasets: (1) a single-step MLP with frame stacking; (2) an MLP policy retrained using DAgger [5]; and (3) the ACT architecture [7].

Our **proposed methods** include: (1) retraining ACT with DAgger using chunked data collection, and (2) the Residual Chunk Blending algorithm. We then evaluate changes in success rate and failure modes under human intervention, comparing performance across algorithms.

We implemented all policies from scratch with the help of generative AI, using Python libraries including PyTorch and NumPy. We collected human data for data augmentation and visualized training results using the existing `gym-pusht`

environment (`gym_pusht/PushT-v0`) for the PushT task. We evaluated our models based on its success rate and its ability to make small adjustments when the difference between the T object and the T goal is minimal.

3.1 Single-Step MLP with Frame Stacking

Our first baseline policy maps each observed state to an action, and was trained with a combination of ResNet and MLP for visual and state inputs. To mitigate the standard frame-by-frame BC's inability to infer dynamic information, such as object velocities, we implemented a frame-stacking mechanism. Specifically, the policy network processes a fused input of visual observations and the 2D coordinates of the robot. For a given time step t, the visual input consists of the current camera frame and a delayed past frame at $t - 9$ (captured at 10 Hz), which are concatenated along the channel dimension to form tensor inputs.

We leveraged a pre-trained ResNet-18 backbone (using weights from ImageNet) to process the 96x96 visual frame inputs. To accommodate the stacked frames, the weights of the first convolutional layer were modified to accept a 6-channel input (3 color channels for each frame) while preserving the pre-trained feature extraction capabilities for the remaining layers. The ResNet backbone outputs a 512-dimensional visual feature vector.

Concurrently, the corresponding robot coordinate states are encoded via a two-layer Multi-Layer Perceptron (MLP) into a 64-dimensional state feature vector. The concatenated visual and state features are passed through a 3-layer ReLU MLP (with hidden size 256) to predict a 2D continuous action.

3.1.1 Training Process and Hyperparameters

Loss Function We first used Mean Squared Error (MSE) as our loss function, but the agent exhibited severe mode-averaging. Because PushT demonstrations contain multimodal solutions for similar states, MSE minimized the squared penalty by predicting the mathematical mean of these actions, resulting in invalid, non-expert trajectories, where the end-effector flies into the environment boundary and remain pinned.
We then switched the loss function to the *Smooth L1 loss* with $\beta = 1$, as follows.

$$L(y, \hat{y}) = \begin{cases} 0.5(y - \hat{y})^2 & \text{if } |y - \hat{y}| < 1 \\ |y - \hat{y}| - 0.5 & \text{otherwise} \end{cases} \tag{1}$$

Since this loss function approximates the median of the action distribution, it more successfully preserved the multi-step dynamics of the task. As a result, the agent was able to approach and withdraw from the T-block continuously. This shows that bounding the gradient penalty for outliers is necessary to prevent catastrophic extrapolation errors in continuous control tasks.

Hyperparameters and Optimizer We used an AdamW optimizer with weight decay 10^{-5}. We trained for 50 epochs, with batch size 64 and learning rate 10^{-4}.

3.2 ACT

We implemented ACT following the architecture of the original ACT paper [7], with several modifications specific to the PushT setting.

Section 2.3 provides a high-level overview of Action Chunking with Transformers (ACT). Here, we describe the exact architecture and implementation details used in our experiments so that our results are fully reproducible.

3.2.1 CNN ResNet Backbone

For the image-only dataset, the image data was processed using a ResNet backbone before being fed into the transformer.

A major difference between the ACT paper's CNN and our experiment's was that the image data in the task of the ACT paper consisted of four 640×480 images. This is downsampled by a factor of 32 in each dimension using ResNet and then flattened, resulting in

$$4 \cdot \frac{640}{32} \cdot \frac{480}{32} = 4 \cdot 20 \cdot 15 = 1200$$

image tokens. On the other hand, each image data point from `lerobot/pusht` is just one 96×96 image, which means our ResNet outputs only

$$\frac{96}{32} \cdot \frac{96}{32} = 3 \cdot 3 = 9$$

tokens for the CVAE decoder to use. This is significantly fewer tokens than the original paper, and we suspected that this could cause the model to suffer in performance despite the relative simplicity of the pushT task.

3.2.2 CVAE with Transformer Encoder/Decoder

For the image-only dataset, we pass the images into the ResNet backbone, then pass the resulting tokens and agent position states from the dataset into a CVAE with transformer encoder/decoder backbone, where each observation consists of the 9 tokens from ResNet and a 2D agent position state.

For the state-only dataset, we directly pass the data into the transformer, where each observation consists of an 18-dimensional state, decomposed into a 2D agent position and a 16D environment representation corresponding to keypoints of the T-block. Rather than treating this as a single flat vector, we follow the LeRobot implementation and embed these as two separate tokens, each linearly projected into the model hidden dimension.

Actions are 2-dimensional continuous coordinates (target (x, y)), and the model predicts a chunk of $chunk_size$ future actions at each timestep.

The CVAE encoder is a Transformer encoder that operates on the sequence $[\text{CLS}, \text{state token}, \text{action chunk}]$, where sinusoidal positional encodings are applied to the action tokens. It outputs the mean and log-variance of a latent variable $z \in \mathbb{R}^{32}$, which is sampled via the reparameterization trick and projected into a latent token. In our implementation, only the agent state (2D) is used as input to the CVAE encoder, while the full observation is used in the decoder context.

The CVAE decoder consists of a Transformer encoder/decoder pair that takes K learned query embeddings (one per timestep in the action chunk) and attends over the encoded observation tokens. The output is projected through a linear layer to produce a $K \times 2$ action sequence. At test time, the model predicts an action chunk at each timestep. We execute a temporally ensembled action, where overlapping predictions are combined using exponential weighting to improve smoothness and reduce error.

3.2.3 Training Procedure

Image-Only Dataset We trained this model for 500 epochs on an NVIDIA GeForce RTX™ 3050 GPU. We used chunk size 20, validation split 0.1, batch size 64, hidden dimension 512, feedforward dimension 2048, 8 attention heads, latent dimension 32, 4 encoder layers, 7 decoder layers, a KL beta of 10 (as the ACT authors recommended), a weight decay parameter of 10^{-5}, GeLU activation function, and a learning rate of 10^{-4} (using the AdamW optimizer). Note that our loss function is still smooth L1. This took roughly 8 hours of training time.
We decreased the KL β to 0.01 or even 0 to increase model complexity, as the KL divergence collapsed to nearly zero when $\beta = 10$. However, the model would then often become stuck, possibly due to excessive complexity. We also experimented with different encoder/decoder configurations and learning rates; the hyperparameter set above yielded the best performance.

State-Only Dataset We first used the same hyper-parameters as the first dataset to compare across the two datasets. After the comparison results, we did marginal changes to improve the performance, such as decreasing learning rate so that validation loss stabilizes better, and matching feedforward dimension and decoder layers to LeRobot ACT model hyper-parameter suggestion.
At last, we trained our model at: Chunk size 16, Feedforward dimension 3200, 4 decoder layers, a weight decay parameter of 10^{-4}, and a learning rate of $3 \cdot 10^{-5}$, with all other hyper-parameters unchanged.

3.3 DAgger

Our **proposed** data aggregation is slightly different from the original DAgger, in the way that each iteration of the policy, the model keeps generating its own state-action pairs until human expert steps in when the model is not performing as expected, and then human finishes the task. The model's moves connected with human's correction moves are saved in a whole trajectory every rollout. Then, during retraining, since the intervention dataset is much smaller than the original dataset, a simple merge would result in the model ignoring our new corrections, so we forced the data loader to sample human corrections vs. original dataset using given ratios (usually half-half), ensuring the model actually learns the recovery behaviors without catastrophically forgetting the original task execution.

3.3.1 Residual Blending

Inspired by compliant residual DAgger [6], we adopt the idea of learning from "delta corrections," allowing a policy to execute under human guidance. While CR-Dagger implements this using force-based actions and kinematic equations, our *Residual Blending* algorithm uses a simpler weighted combination of the base policy and human corrections, where human corrections at any point are taken as incremental corrections added onto model predictions, resulting in action $a_{blended}$:

$$a_{blended} = (1 - \lambda) \cdot a_{base} + \lambda \cdot a_{human} \tag{2}$$

where λ is a hyperparameter representing the weight of the human intervention.

Furthermore, instead of using CR-DAgger residual policy, we opt to fine tune our original policy, focusing on simplicity and computational efficiency.

4 Experimental Results

4.1 Evaluation Metrics

We evaluate each policy using the following metrics:

1. **Success rate**
2. **Percent of pushing to dead zones**: Cases where the model pushes the T into regions with little to no training data (typically near environment boundaries), becomes stuck, and fails to recover.
3. **Percent of stagnation**: Cases where the model fails to select a next action and remains stationary.
4. **Percent of near-misses**: Cases where the model approaches the target closely but fails to fully fit into it.
5. **Percent of aggressive near-fixes**: Cases where the model largely places the T into the goal region but subsequently perturbs it in a way it cannot recover from.

Metrics 2-5 were motivated by observations of how the trained models failed in the task. Note that we evaluated each of our methods for 30-50 trials.

4.2 Image-Only Dataset on Baseline Methods

4.2.1 Single-Step MLP, MLP & DAgger

For the first baseline policy, our trained model was unable to push the T at all. Instead, it would just move slowly towards the T, eventually converging to a point adjacent to the T and stopping. We attribute this to a combination of the model's single-step prediction behavior (which lacks temporal context) and aggressive downsampling by ResNet of the already low-quality input frame. As a result, the agent cannot form long-term goals, which are necessary for a long-horizon planning-intensive task such as PushT.

Our second baseline policy, single-step MLP with DAgger, helped mitigate the most obvious problem of not even touching the T, as the agent was essentially being "taught" to make contact with the T when it was near. However, this policy likewise achieved a 0% success rate, since even though the agent was capable of contacting the T, its inability to plan prevented meaningful task progress.

4.2.2 ACT

After several iterations of hyperparameter-tuning, the performance of our ACT-based model was much better than the per-step MLP model. After just 20 epochs of training, the agent was able to successfully make and break contact with the T and push it to roughly the correct location. However, the actual success rate (operationalized as 90% target zone overlap) was still very low. The distribution of results contains multiple failure modes, as shown in Table 1.

Table 1: Dataset 1 ACT Results

Success Rate	Near-misses	Stagnation	Push to Border	Aggressive Fix
10%	24%	32%	14%	30%

These failure modes suggest several underlying limitations. *Aggressive near-fix* failures indicate a lack of a termination or stabilization policy once the goal is nearly achieved. *Push to border* (dead-zone) failures reveals there are under-represented states that led to poor generalization and an inability to recover. *Stagnation* reflects uncertainty in action selection, likely due to insufficient data coverage or ambiguity in similar states, causing the policy to output minimal or no movement. *Near-misses* indicate that while the model captures coarse spatial alignment, it lacks the precision required for final placement.

A likely explanation for the persistently low success rate is the limited size and coverage of the training dataset. Compared to the datasets and tasks used in the original paper, wherein the input is significantly richer and covers more of the state space, the Push-T dataset we use provides lower-quality examples of successful long-horizon behaviors

and recovery from suboptimal states, constraining the model's ability to learn fine-grained control. On the bright side, we compared our results with quite a few online ACT models trained on the same dataset, such as a public model on HuggingFace, and most of them had only 1-3 successes out of 50 trials, with lots of aggressive near-fixes (thus less near-misses), and similar amount of stagnation and pushing to border behavior, which proved the effectiveness of the hyper-parameters our model ends up using. Hence, though surprising to us due to ACT's high performance in its paper, it could be scientifically concluded that ACT, trained on the image-only dataset, does not perform well in Push-T in general.

4.3 State-Only Dataset on All Methods

Since running the previous dataset did not reach a desirable result on the baseline methods, we directly proceeded onto training on state-only dataset (`lerobot/pusht_keypoints`) for direct comparison, and then attempting fine-tuning through proposed methods, to more deeply investigate human intervention's actual effect on MLP and ACT.

4.3.1 Single-Step MLP, MLP & DAgger

Table 2: Dataset 2 Single-Step MLP Results

Success	Near-miss	Stagnation	Push to Border
6%	9%	5%	80%

The MLP with DAgger has similar results as the MLP without it because DAgger only addresses the problem of distributional shift during model training. The policy remains a single-step predictor, unable to plan ahead. Still, as we expected, while it helps the original model recover from the most obvious failure mode of not making contact with the T, it cannot fix the fundamental problem of model architecture being unfit for the pushT task.

4.4 ACT

We compared effects after 30, 150, and 250 epochs.

Table 3: Dataset 2 ACT Results after 30, 150, 250 Epochs

No. of Epochs	Success Rate	Near-misses	Stagnation	Push to Border	Aggressive Fix
30	$10 \pm 3\%$	$54 \pm 3\%$	$60 \pm 2\%$	$0 \pm 0\%$	$10 \pm 2\%$
150	$40 \pm 5\%$	$45 \pm 3\%$	$33 \pm 3\%$	$0 \pm 0\%$	$4 \pm 0\%$
250	$50 \pm 2\%$	$42 \pm 3\%$	$30 \pm 3\%$	$0 \pm 0\%$	$4 \pm 0\%$

Although the validation loss stabilizes after approximately 30 epochs (Figure 5), performance continues to improve significantly with further training. This suggests that the validation loss does not fully capture the model's "ability to push the T," as the loss simply measures the model's accuracy in imitation of expert trajectories. In particular, later training stages improve the agent's ability to perform fine-grained corrections when the T is close to the goal, while models trained for fewer epochs tend to stagnate near the goal state, failing to complete the final alignment. The better result for longer training aligned with the ACT paper's advice that empirically one should train for long after validation validation loss stabilizes.

Crucially, this improvement compared to the first dataset occurred even without changes to the model architecture or hyperparameters, highlighting the impact of the state-only dataset. Compared to the image-based setting, the keypoint representation provides a more precise and structured description of the T block, allowing for better generalization.

4.5 ACT & DAgger/Residual Blending

At the same time, we took our best policy and did data augmentation on it using DAgger and residual blending, respectively, the results are as follows (table 4).

We can see from the table that both messages completely eliminated the "aggressive fix" failure modes. This likely because both methods counteract bad model behavior, disabling such high-speed movements of the end effector. However, this counteraction also seems to result in higher stagnation due to the human intervention often directly opposing incorrect model behavior.

Table 4: ACT with DAgger vs. Residual Blending

Model	Success Rate	Near-misses	Stagnation	Push to Border	Aggressive Fix
ACT + DAgger	30%	66%	56%	0%	0%
ACT + Residual Blending	10%	58%	76%	0%	0%

5 Discussion

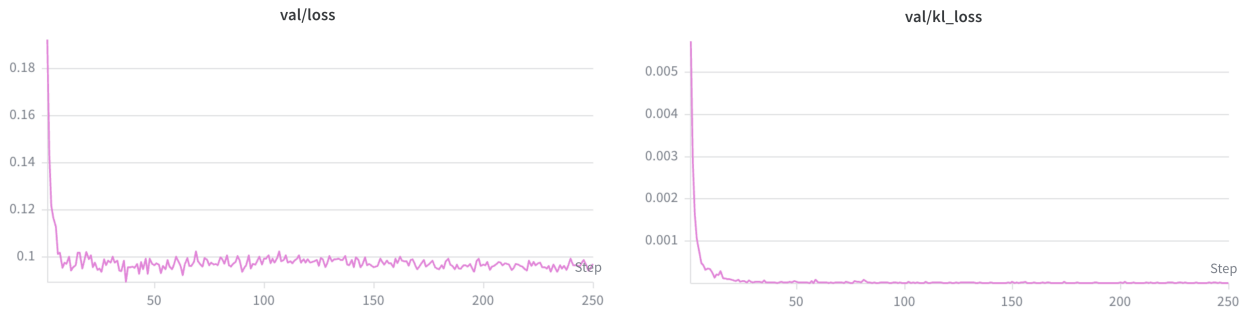


Figure 5: Validation loss curve (left) and KL loss curve (right) for ACT training on state-only dataset. The x-axis represents the number of epochs, and the y-axis represents the loss value at each epoch.

Through experiments of the two datasets, we reasonably conclude that the image-only input type for the position of the T block and the downsampling of the ResNet backbone introduce a representational bottleneck that limits the quality of visual features. This degradation appears to propagate into the CVAE, where the model increasingly ignores the latent variable z. Empirically, for $\beta > 0.001$, we found that the $\mathcal{L}_{reg}$ would quickly converge to zero. This seems to suggest that the ACT model tends to not learn a latent representation at all when training on the image-only dataset. We believe this can be attributed to the low quality of images we were using: we used 96×96 images, while the original ACT paper's task used four 640×480 images.

Even when training on the 18-dimensional state-only input from `lerobot/pusht_keypoints`, the limited input dimensionality led the model to rely heavily on observations for predictions, ignoring the style variable. We can see the model learning to do this in Figure 5: The KL loss collapses to zero, meaning that the latent space encodes nearly no information. Furthermore, the results of DAgger and Residual Blending suggest that naive integration of human corrections can degrade performance for action chunking policies, resulting in stagnation. Consider a failure mode in which the model is pushing the T toward the right far more than it should. The human action in this case would be the complete opposite, with the goal of moving the T back towards the left. For the Residual Blending algorithm, this would results in nearly no movement at this state since human and machine movement cancels out.

For DAgger, we observe a related but unique failure mode. While it improves coverage of off-distribution states, it introduces conflicting signals in regions where the base policy previously relied on consistent (even if suboptimal) behavior. As a result, performance can decrease in states where the model previously succeeded without following expert trajectories, since the aggregated dataset now contains both corrective and non-corrective trajectories, increasing ambiguity. Furthermore, the mode of the new data collected is focused on recovery, but the original data focused on solving the task, so the difference in modes caused data imbalance, which hinders ACT's performance. The earlier discussed kl divergence collapse further exacerbates this problem, as the policy does not use its z value to differentiate between these two successful chunks.

6 Data and Code

The two dataset we used are: textttlerobot/pusht and `lerobot/pusht_keypoints`. This is the linke to our GitHub repository, where the `data` folder contains the manually collected data for DAgger and residual blending.

References

[1] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion, 2024.

[2] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling, 2025.

[3] Lerobot. PushT Dataset. `https://huggingface.co/datasets/lerobot/pusht`. Hugging Face Datasets.

[4] Lerobot. PushT Keypoints Dataset. `https://huggingface.co/datasets/lerobot/pusht_keypoints`. Hugging Face Datasets.

[5] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning, 2011.

[6] Xiaomeng Xu, Yifan Hou, Chendong Xin, Zeyi Liu, and Shuran Song. Compliant residual dagger: Improving real-world contact-rich manipulation with human corrections, 2025.

[7] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware, 2023.